03 — Architecture Explanation

This document explains how SeatMe is built and why. The canonical, always-current version (with Mermaid sequence diagrams) lives at ../../docs/architecture.md; this is a self-contained summary for the submission.

1. Overview

SeatMe is **fully serverless**: every component is an AWS managed service, so there are no servers to provision, patch, or scale by hand, and the application scales to zero when idle. The browser is a thin client that talks to two independent backends — **Cognito** for identity and **API Gateway → Lambda → DynamoDB** for data.

2. Components and responsibilities

Component	Service	Responsibility
Frontend	Amazon S3 (static website)	Serves the HTML/CSS/JS screens
Auth	Amazon Cognito User Pool	Sign-up, email verification, sign-in, password reset, group membership
API	Amazon API Gateway (HTTP API)	Routes JSON requests to Lambda
Compute	AWS Lambda (Python 3.12)	One function per operation; validation + one DynamoDB op
Database	Amazon DynamoDB	Single table, host-centric design
Email	Amazon SNS	Sends guests their personal RSVP link

3. Why these choices

- **S3 static hosting** — the frontend is plain HTML/CSS/JS (no build step, no npm), so S3 is the cheapest and simplest way to serve it globally.
- **Cognito** — provides secure identity (hashed passwords, email verification, token issuance, password reset) without writing any auth code, and **user groups** give us the required two permission tiers (`host` and `admin`).
- **API Gateway HTTP API** — lower cost and latency than REST API, and a clean mapping of one route per operation.
- **Lambda** — each operation is a small, independently deployable function; pay only per invocation; no idle cost.
- **DynamoDB** — single-digit-millisecond key-value access that fits the host-centric data model, with `PAY_PER_REQUEST` so there is no capacity to plan.
- **SNS** — managed fan-out email delivery for invitations.

4. Data model (single table, host-centric)

The table `SeatMe` has one partition key `email` (the host's email). **Everything about an event — its guests and tables — lives inside one item**, so opening a host screen is a single `GetItem`.

```
{
  "email": "host@example.com",
  "name": "Jane Cohen",
  "event_name": "Jane & Tom's Wedding",
  "event_date": "2026-09-15",
  "event_location": "Tel Aviv",
  "categories": ["Family", "Friends", "Work"],
  "tables": { "1": { "capacity": 10 }, "2": { "capacity": 8 } },
  "guests": {
    "daniel@example.com": { "name": "Daniel Levi", "category": "Family",
      "count": 2, "rsvp": "yes", "table": 1 }
  }
}
```

> **Numbers as strings.** `get_host` / `get_guests` serialize with `json.dumps(default=str)`, so DynamoDB `Decimal` values arrive as strings; the frontend `parseInt`s them.

5. Request flow (sign in → open event)

1. Browser calls Cognito `InitiateAuth` (`USER_PASSWORD_AUTH`) and stores the Id/Access/Refresh tokens in `localStorage`.
2. Browser calls `GET /hosts?host_email=...` on API Gateway.
3. API Gateway invokes `get_host`, which does a single `GetItem` and returns the event summary as JSON.

Each Lambda (a) parses and validates input, (b) performs one DynamoDB operation — usually a conditional update — and (c) returns `{ statusCode, body }`.

6. Seating algorithm

`generate_seating` is greedy and category-aware: filter to `yes` guests, check total capacity, group by category (largest first), place each group into the table with the most free seats so categories stay together, then persist all assignments in one write.

7. Invitations (SNS)

SNS has no "email one address" API, so each guest is modeled as a **topic subscription with a filter policy on their own email**. The first email is a one-time *Confirm subscription* message; once confirmed, publishes (carrying a matching `guest_email` attribute) deliver only that guest's personal link.

8. Permission model (two groups)

- **host** — the default group for event organizers; new sign-ups are auto-added to it by a Cognito **Post-Confirmation Lambda trigger** (`auth_post_confirmation`). Members manage **their own** event only.

- **admin** — can list and manage **all** events via `GET /admin/hosts` and the admin portal. Because the HTTP API is intentionally open (so the public RSVP and the no-login preview link work), the admin route is enforced **inside the Lambda**: it validates the caller's Cognito access token (`GetUser`) and checks `admin` group membership (`AdminListGroupsForUser`) before returning data.
- **Per-request ownership checks** — every **write** endpoint (create / update / delete a host or guest, set tables, generate seating, send invitations) validates the caller's Cognito access token and confirms the caller either **owns** the targeted event (token email == host email) or is an **admin**, via a shared `_common.require_owner` helper bundled into each Lambda. Requests that arrive without an API Gateway request context (the direct SDK invokes used by the seed script) are treated as trusted server-side calls and bypass the check.

9. Deployment model

`redeploy_all.py` is the single entry point and reuses the same helper modules used for manual deploys:

```
redeploy_all.py
├── backend/deploy/setup_aws.py       → Lambdas + API Gateway
├── backend/deploy/setup_cognito.py   → Cognito user pool + client + groups
+ seed users
├── backend/deploy/seed_example.py    → optional demo data
└── frontend/deploy_frontend.py       → upload site to S3 (+ inject config)
```

At deploy time, `deploy_frontend.py` injects the API URL and Cognito IDs into the HTML/JS by replacing `REPLACE_WITH_*` placeholders, so **no environment URLs or secrets are committed**.

10. Security notes

- **Writes are authenticated and authorized**: mutating endpoints require a valid Cognito access token and enforce per-event ownership (or `admin`) inside the Lambda (`_common.require_owner`); the frontend attaches `Authorization: Bearer <access token>` to every write call.
- **Reads stay public by design**: `get_host` / `get_guests` / `rsvp_guest` are unauthenticated so the public RSVP page (F15) and the no-login preview link (F19) keep working. An anonymous preview is therefore **read-only** — the UI shows a read-only banner and disables editing, and the server rejects any write it attempts.
- Input validation and DynamoDB **condition expressions** keep data consistent; party size is capped (1–20) and stale seat assignments are cleared when tables or seating change.
- The frontend **escapes** all user-supplied values (`escapeHtml`) to prevent XSS, and 500 responses no longer echo internal exception detail.
- No secrets in the repo (placeholders injected at deploy time).
- **Known limitation / roadmap:** attach a Cognito JWT authorizer to the whole HTTP API to move token validation off the Lambda code path (today each protected Lambda validates the caller's token itself).